

Primer programa
Analizador Léxico**Objetivo**

Elaborar un analizador léxico en *lex/flex* que reconozca los componentes léxicos pertenecientes a las clases abajo descritas y que fueron definidas en clase.

Descripción

- Las clases de los componentes léxicos válidos para el analizador léxico son:

Clase	Descripción
0	Palabras reservadas (ver tabla).
1	Identificadores. Inician con “I_ “ y terminan con “_I”. Puede contener dígitos del 0 al 9, letras mayúsculas y minúsculas, y “_”. El tamaño de la cadena no debe rebasar los 32 caracteres, contando los iniciales y los finales. Ejemplos: I_VariableNum1 _I I_valor_mayor_I
2	Operadores de asignación (ver tabla).
3	Operadores relacionales (ver tabla)
4	Operadores aritméticos (ver tabla)
5	Símbolos especiales (ver tabla)
6	Constantes enteras. Contiene dígitos del 0 al 9 y terminan con i. Se usa “n” o “p” para indicar número negativo o positivo respectivamente. Ejemplos: p25i n55i 0i. El valor 0 sólo se representará como 0i
7	Constantes reales. Se podrán representar simples con “.” decimal o bien con notación científica: Ejemplos: 1.1 .24 -2.2 32e-3 -4.4E2
8	Constantes cadena. Inician con < y terminan con >. La cadena puede contener 0 o más caracteres cuyo ASCII esté entre 32 y 126 inclusive. Se puede utilizar @n, @f, @d, @t para referenciar un valor constante o un valor asignado a una variable.

El número de clase es inamovible.

- El analizador léxico tendrá como entrada un archivo con el programa fuente, **el cual se indicará desde la línea de comandos** al momento de mandar a ejecutar el analizador léxico.
- Como delimitador de un componente léxico será uno o varios espacios, tabuladores o saltos de línea, **así como el inicio de otro componente léxico**. Considerar el orden en las acciones en el programa *lex/flex*
- Los tokens se representarán en una estructura con dos campos:
campo1: la clase
campo2: el valor (de acuerdo con las siguientes tablas e indicaciones)

Valor	Palabra reservada	Equiv	Valor	Palabra reservada	Equiv
0	bool	boolean	9	if	if
1	cad	cadena	10	int	int
2	cte	const	11	leer	leer
3	def	funciones	12	mientras	while
4	decisión	switch	13	mostrar	escribe
5	elegir	case	14	para	for
6	flot	float	15	regresa	return
7	fls	false	16	vdd	true
8	haz	do			

Valor	Símbolos especiales	Equiv
0	@>	
1	\$\$	
2	#?	
3	::=	
4	?=	
5	%~	
6	+	

Valor	Op. asignación
0	:=
1	:+
2	:-
3	:*
4	:/
5	:%

Valor	Op. relacional	Representa
0	+q	Mayor que
1	-q	Menor que
2	+iq	Mayor o igual
3	-iq	Menor o igual
4	:i	Igual que
5	d/i	Diferente a

Valor	Op. aritmético	Representa
0	&+	suma
1	&-	resta
2	&*	multiplica
3	&/	división
4	&&*	potencia
5	&%&	residuo
6	&*&	raíz cuadrada

El valor para el token de cada identificador es la posición dentro de la tabla de símbolos; para ello deberá buscar si el identificador ya está en la tabla; en caso de no estar lo insertará en la tabla. Para las palabras reservadas, los operadores aritméticos, los operadores relacionales, los símbolos especiales y los operadores de asignación, el valor en su token será la posición en su tabla correspondiente (catálogo).

Manejará dos tablas de literales, una para las constantes cadena y otra para las constantes reales; por lo que el valor de su token será la posición dentro de su correspondiente tabla. Cada vez que el Analizador Léxico encuentre una cadena o una constante real NO revisará si ya se encuentra en su tabla de literales, simplemente la insertará.

Las constantes enteras como valor de su token será el valor numérico de la constante entera.

- El analizador deberá crear la Tabla de Símbolos para almacenar a los identificadores. Esta tabla manejará los campos: posición, nombre del identificador y tipo (este último será de tipo entero y podrán ponerle como valor inicial -1). Se indicará en el documento a entregar, el tipo de estructura de datos empleada, así como el método de búsqueda a utilizar.

- Las tablas de literales para las cadenas y para las constantes reales, deberán tener como estructura, dos campos: la posición y el dato (como cadena).
- El Analizador Léxico deberá reconocer los comentarios y descartarlos. Los comentarios de línea se identifican porque inician con `“//”`. Los comentarios de varias líneas inician con `/*` y terminan con `*/`.
- Al término del análisis léxico deberá mostrar la tabla de símbolos, las tablas de literales, así como la secuencia de tokens. También podrán almacenarse en archivos para su mejor revisión.
- Los errores que vaya encontrando el analizador léxico, los podrá ir mostrando en pantalla o escribirlos en un archivo, indicando en qué línea del programa fuente lo encontró. Deberá recuperarse de los errores encontrados para continuar con el reconocimiento de todos los componentes léxicos del archivo de entrada.
- El programa deberá estar comentado, con una descripción breve de lo que hace (puede ser el objetivo indicado en este documento), el nombre de quién(es) elaboró(aron) el programa y fecha de elaboración, así como lo que hace cada función. Se deberá cuidar mucho la sangría que denota la dependencia de instrucciones.

Entregar:

Un documento con la siguiente estructura:

- Descripción del problema (no del programa), incluyendo características que deben cumplir los componentes léxicos que reconocerá el analizador léxico y **la expresión regular de cada clase**. No transcribir lo indicado en este documento.
- Propuesta de solución y fases del desarrollo del sistema: Análisis (planificación, indicando participantes por cada actividad), Diseño y Pruebas. En el diseño indicar exactamente cómo se definirán la tabla de símbolos, las tablas de literales y los tokens, la técnica de búsqueda e inserción de los identificadores. (No poner sólo el código de la definición de las estructuras de datos).
- Indicaciones de cómo correr el programa.
- Conclusiones por cada participante.

Nota: se podrá elaborar individualmente o en equipo de 2

Enviar el documento y sólo el programa fuente definitivo a la plataforma educativa (apartado tareas y empaquetados en .rar o .zip) y en su caso, **sólo un miembro del equipo**.

Fecha de entrega: 29 de septiembre de 2026.